

# EVIDENCIAS — QUESTAO 2

Chatbot com IA Generativa | LangChain + GPT-4

Prova Tecnica — Dot Digital Group

Candidato: Silas Luiz Bom Fim | 2025

## 1. Objetivo da Questao

Desenvolver um chatbot baseado em IA Generativa utilizando **LangChain** e o modelo **GPT-4** da OpenAI. O chatbot deve manter historico de conversa entre as mensagens, responder perguntas sobre programacao Python em portugues brasileiro e demonstrar raciocinio avancado ao lidar com problemas complexos.

**DESTAQUE: Os testes locais foram realizados com problemas de Calculo Numerico — disciplina universitaria de alta complexidade — para demonstrar que o chatbot vai muito alem de respostas superficiais sobre Python.**

## 2. Tecnologias Utilizadas

|                           |                                                         |
|---------------------------|---------------------------------------------------------|
| Linguagem                 | Python 3.10+                                            |
| Framework de IA           | LangChain + langchain-openai                            |
| Modelo de Linguagem       | GPT-4 (OpenAI)                                          |
| Gerenciamento de Segredos | python-dotenv (.env local — nunca versionado)           |
| Historico de Conversa     | SystemMessage, HumanMessage, AIMessage (LangChain Core) |
| Execucao                  | Terminal interativo (python main.py)                    |

## 3. Estrutura doCodigo

|                  |                                                                             |
|------------------|-----------------------------------------------------------------------------|
| main.py          | Logica principal: conexao ao GPT-4, historico de mensagens, loop interativo |
| requirements.txt | Dependencias: langchain, langchain-openai, python-dotenv                    |

|                           |                                                                    |
|---------------------------|--------------------------------------------------------------------|
| <code>.env (local)</code> | Chave OPENAI_API_KEY — nunca versionada, protegida pelo .gitignore |
| <code>.env.example</code> | Modelo de configuracao com placeholder 'sua_chave_aqui'            |
| <code>README.md</code>    | Instrucoes de instalacao, configuracao e execucao do chatbot       |

## 4. Codigo-Fonte Principal (main.py)

O arquivo **main.py** concentra toda a logica do chatbot. Cada parte foi comentada de forma didatica para facilitar a compreensao:

```

import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

# Carrega as variaveis de ambiente do arquivo .env
load_dotenv()

# Le a chave da API da OpenAI a partir do .env
api_key = os.getenv("OPENAI_API_KEY")

# Conecta ao modelo GPT-4 da OpenAI usando a chave carregada
llm = ChatOpenAI(model="gpt-4", api_key=api_key)

# Mensagem de sistema – define o papel e o comportamento do chatbot
system_message = SystemMessage(
    content=(
        "Voce e um assistente especialista em programacao Python. "
        "Responda sempre em portugues brasileiro, de forma clara e didatica, "
        "com exemplos de codigo quando necessario."
    )
)

# Historico da conversa – comeca apenas com a mensagem de sistema
historico = [system_message]

def chat(pergunta: str) -> str:
    # Adiciona a pergunta do usuario ao historico
    historico.append(HumanMessage(content=pergunta))

    # Envia todo o historico ao modelo e obtem a resposta
    resposta = llm.invoke(historico)

    # Adiciona a resposta do modelo ao historico para manter o contexto
    historico.append(AIMessage(content=resposta.content))

    return resposta.content

if __name__ == "__main__":
    print("=== Chatbot Python ===")
    print("Especialista em programacao Python.")
    print("Digite 'sair' para encerrar.\n")

    while True:
        pergunta = input("Voce: ").strip()
        if not pergunta:
            continue
        if pergunta.lower() == "sair":
            print("Encerrando o chatbot. Ate logo!")
            break
        resposta = chat(pergunta)
        print(f"\nChatbot: {resposta}\n")

```

## 5. Testes Locais — Calculo Numerico

Os testes foram realizados com 4 problemas reais de Calculo Numerico — disciplina avancada da graduacao em Engenharia/Ciencia da Computacao. Isso comprova que o chatbot e capaz de resolver desafios matematicos e computacionais complexos, nao apenas responder perguntas basicas sobre Python.

Ambiente de execucao registrado no terminal (Windows 11):

```
Microsoft Windows [versao 10.0.26200.8655]
C:\Users\silas> cd C:\Users\silas\OneDrive\Desktop\dot-backend-test\q2_chatbot
C:\...\q2_chatbot> python main.py
=== Chatbot Python ===
Especialista em programacao Python.
Digite 'sair' para encerrar.
```

### Teste 1 — Analise de Sistemas Lineares (Algebra Linear)

Problema classico de Calculo Numerico: dado um sistema linear  $Ax = b$ , determinar se ele possui unica solucao, infinitas solucoes ou nenhuma solucao. O chatbot aplicou o Teorema de Rouche-Capelli usando **NumPy** e o conceito de **rank** (posto) de matrizes.

Pergunta enviada ao chatbot:

```
escreva um programa em python que faz uma analise de um sistema linear
e determina o numero de solucoes
```

Codigo gerado pelo chatbot:

```
import numpy as np

def numero_solucoes(A, b):
    A = np.array(A)
    b = np.array(b)
    rank_A = np.linalg.matrix_rank(A)
    Ab = np.concatenate((A, b), axis=1)
    rank_Ab = np.linalg.matrix_rank(Ab)
    n = len(A[0])
    if rank_A == rank_Ab:
        if rank_A == n:
            return 'Sistema possui unica solucao'
        else:
            return 'Sistema possui infinitas solucoes'
    else:
        return 'Sistema nao possui solucao'

A = [[2, 3], [4, 6]]
b = [[6], [12]]
print(numero_solucoes(A, b))
```

**Resultado:** O chatbot implementou corretamente o Teorema de Rouche-Capelli, comparando o posto da matriz dos coeficientes com a matriz aumentada.

## Teste 2 — Metodo de Newton-Raphson (Calculo de Raizes)

O Metodo de Newton-Raphson e um dos algoritmos mais utilizados em Calculo Numerico para encontrar raizes de funcoes nao-lineares de forma iterativa. O chatbot utilizou a biblioteca **SymPy** para calcular a derivada simbolica da funcao e implementou o loop iterativo com criterio de parada por tolerancia (epsilon).

### Pergunta enviada ao chatbot:

```
quero um programa em python para calcular cada um dos valores
utilizando o metodo de Newton-Raphson
```

### Codigo gerado pelo chatbot:

```
import sympy as sp

x = sp.symbols('x')
f = x**3 - x**2 + 2
f_prime = sp.diff(f, x)

def newton_raphson(f, f_prime, x0, epsilon=1e-10, max_iter=100):
    x = x0
    for _ in range(max_iter):
        x_old = x
        x = x - f.subs('x', x)/f_prime.subs('x', x)
        if abs(x - x_old) < epsilon:
            break
    return x

x0 = -20
root = newton_raphson(f, f_prime, x0)
print(f'Raiz encontrada: {root}')
```

**Resultado: O chatbot implementou Newton-Raphson com derivada simbolica (SymPy), criterio de convergencia e limite de iteracoes — solucao robusta e correta.**

## 5. Testes Locais — Calculo Numerico (continuacao)

### Teste 3 — Metodo de Gauss-Seidel com Criterio das Linhas

O Metodo de Gauss-Seidel e um algoritmo iterativo para solucao de sistemas lineares. Este teste foi o mais complexo ate entao: alem de implementar o metodo, o chatbot verificou o **Criterio das Linhas** (condicao suficiente de convergencia) antes de executar as iteracoes — comportamento esperado de uma implementacao profissional.

#### Pergunta enviada ao chatbot:

Agora eu quero um programa em python para resolver o sistema linear dado usando o metodo de Gauss-Seidel, verificando se o criterio das linhas e satisfeito antes de continuar o algoritimo

#### Codigo gerado pelo chatbot:

```
import numpy as np

def verifica_criterio_linhas(A):
    n = len(A)
    for i in range(n):
        soma = sum(abs(A[i, j]) for j in range(n) if i != j)
        if abs(A[i, i]) <= soma:
            return False
    return True

def gauss_seidel(A, b, x0, max_iter=100):
    if not verifica_criterio_linhas(A):
        print('O sistema nao satisfaz o criterio das linhas')
        return
    n = len(A)
    x = x0.copy()
    for it in range(max_iter):
        x_new = x.copy()
        for i in range(n):
            s1 = sum(A[i, j] * x_new[j] for j in range(i))
            s2 = sum(A[i, j] * x[j] for j in range(i + 1, n))
            x_new[i] = (b[i] - s1 - s2) / A[i, i]
        if np.allclose(x, x_new, rtol=1e-8):
            break
        x = x_new
    return x

A = np.array([[4.0, 1.0, 1.0], [2.0, 5.0, 1.0], [1.0, 1.0, 3.0]])
b = np.array([1.0, 1.0, 1.0])
x0 = np.zeros_like(b)
x = gauss_seidel(A, b, x0)
print(f'Solucao: {x}')
```

**Resultado: O chatbot implementou o Criterio das Linhas corretamente e aplicou Gauss-Seidel com criterio de parada `np.allclose` — implementacao de nivel academico.**

### Teste 4 — Integracao Numerica pelo Metodo dos Trapezios

Ultimo teste: calculo da integral definida de  $x^2 * \ln(x)$  de 1 a 2 usando a Regra dos Trapezios com 7 pontos. Este metodo divide a area sob a curva em trapezios e soma suas areas para aproximar a integral. O chatbot explicou o metodo, codificou a solucao com **Numpy** e alertou sobre a natureza aproximada do resultado.

**Pergunta enviada ao chatbot:**

Escreva um programa em Python para calcular a integral de  $x^2 * \ln(x)$  de 1 a 2 usando o Metodo dos Trapezios com 7 pontos.

**Codigo gerado pelo chatbot:**

```
import numpy as np

def f(x):
    return x**2 * np.log(x)

def trapezoidal_rule(a, b, n, func):
    h = (b - a) / (n - 1)
    x_values = np.linspace(a, b, n)
    sum_values = func(a) + func(b) + 2 * sum([func(x) for x in x_values[1:-1]])
    return h / 2 * sum_values

a, b, n = 1, 2, 7
area = trapezoidal_rule(a, b, n, f)
print('A integral de  $x^2 * \ln(x)$  de 1 a 2 com 7 pontos e aprox {:.8f}'.format(area))
```

**Resultado: O chatbot implementou a Regra dos Trapezios corretamente com np.linspace, formula de soma e passo h — resultado preciso para a aproximacao com 7 pontos.**

## 6. Testes via GitHub Codespaces

Alem dos testes locais via CMD, o chatbot foi executado diretamente pelo navegador usando o **GitHub Codespaces** — ambiente VS Code na nuvem integrado ao repositório. Esta etapa comprova que o chatbot funciona em qualquer ambiente, sem depender de instalação local.

**Ambiente: GitHub Codespaces (literate lamp) | Repositorio: silasluiz96-alt/DOT-Project | Branch: main**  
**| Sistema: Linux (Ubuntu) no navegador**

### Teste 5 — Conceitos Basicos de Python para Iniciantes

Neste teste foi avaliada a capacidade do chatbot de orientar um aluno iniciante, listando os principais conceitos de Python com exemplos de código para cada tópico. O chatbot estruturou uma resposta didática cobrindo 7 conceitos fundamentais.

#### Pergunta enviada ao chatbot:

Sou um aluno aprendendo Python, quais os principais conceitos que devo ver primeiro?

#### Resumo da resposta gerada:

1. Sintaxe Basica – variaveis, funcoes, operadores  
`nome = 'Joao'`  
`print('Ola, ' + nome)`
2. Tipos de Dados – int, float, str, list, tuple, dict  
`numero = 10 | decimal = 10.5 | lista = [1,2,3]`
3. Estruturas de Controle – if, else, for, while  
`if idade >= 18: print('Maior de idade')`
4. Funcoes – def, parametros, return  
`def somar(a, b): return a + b`
5. Classes e Objetos – POO, heranca, polimorfismo  
`class Pessoa: def __init__(self, nome, idade): ...`
6. Excecoes – try, except, tratamento de erros  
`try: x = 10/0 except ZeroDivisionError: print('Erro!')`
7. Manipulacao de Arquivos – open, read, write, close  
`arquivo = open('arquivo.txt', 'w')`

**Resultado: O chatbot respondeu de forma estruturada e didática, cobrindo 7 conceitos essenciais com exemplos de código para cada um.**

### Teste 6 — Python para Calculo Numerico e Series de Taylor

Neste teste foi verificado se o chatbot consegue conectar o aprendizado de Python com aplicações matemáticas avançadas. A pergunta abordou Cálculo Numérico e Séries de Taylor, demonstrando continuidade temática com os testes anteriores realizados via CMD.

#### Pergunta enviada ao chatbot:



posso usar python para aprender calculo numerico?  
E equacoes como series de Taylor?

Codigo gerado pelo chatbot:

```
import math

def calculate_taylor_approximation(x, n):
    result = 0
    for i in range(n):
        coeff = (-1) ** i
        num = x ** (2 * i + 1)
        denom = math.factorial(2 * i + 1)
        result += (coeff) * (num / denom)
    return result

x = 1.0
print(calculate_taylor_approximation(x, 10)) # Aprox. de sin(1.0)
```

O chatbot explicou que Python é ideal para Cálculo Numérico graças as bibliotecas **NumPy**, **SciPy** e **matplotlib**, e implementou a aproximação de  $\sin(x)$  usando os primeiros  $n$  termos da Série de Taylor — técnica clássica de Análise Numérica ensinada em cursos de Engenharia e Matemática.

**Resultado: O chatbot demonstrou domínio de Séries de Taylor e sua implementação numérica em Python — confirmando capacidade avançada de raciocínio matemático.**

7. Encerramento da Sessão e Conclusão

```
# Sessão CMD (Windows 11)
Voce: sair
Encerrando o chatbot. Até logo!

# Sessão Codespaces (navegador)
@silasluiz96-alt -> /workspaces/DOT-Project/q2_chatbot (main) $ python main.py
=== Chatbot Python ===
...
```

| Criterio                                       | Status   |
|------------------------------------------------|----------|
| Chatbot funcional com LangChain + GPT-4        | APROVADO |
| Historico de conversa entre mensagens          | APROVADO |
| Respostas em portugues brasileiro              | APROVADO |
| Segredo (API Key) protegido via .env           | APROVADO |
| 4 testes via CMD com Calculo Numerico avancado | APROVADO |
| 2 testes via GitHub Codespaces (navegador)     | APROVADO |

|                                 |                 |
|---------------------------------|-----------------|
| Codigo explicado e didatico     | <b>APROVADO</b> |
| README com instrucoes completas | <b>APROVADO</b> |

**Todos os criterios da Questao 2 foram atendidos com sucesso. Os testes com Calculo Numerico demonstram que o chatbot possui capacidade real de raciocinio e geracao de codigo Python avancado.**